

## 4.1.1. Pandas - series creation and manipulation 02:24

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations.

Hint: You can group the Series based on whether each number is even or odd.

**Input Format:**

- The user should enter a list of numbers separated by space when prompted.

**Output Format:**

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

[Sample Test Cases](#)

seriesMa...

SUBMIT

```
1  """import pandas as pd
2
3  # Take inputs from the user to create a list of numbers
4  numbers = list(map(int, input().split()))
5  """
6
7  # Create a Pandas series from the list of numbers
8  import pandas as pd
9
10 # 1. Take inputs from the user to create a list of numbers
11 # The input is split by spaces and converted to integers
12 numbers = list(map(int, input().split()))
13
14 # 2. Create a Pandas series from the list of numbers
15 series = pd.Series(numbers)
16
17 # 3. Grouping by even and odd numbers and calculating the mean
18 # We group by a condition: if the number % 2 is 0, it's even, else it's
19 # odd.
20 # The index will be True for even and False for odd initially.
21 grouped = series.groupby(series % 2 == 0).mean()
22
23 # 4. Display the mean of even and odd numbers with labels
24 # We map the boolean index (True/False) to 'Even'/'Odd' labels
25 grouped.index = ['Odd' if not is_even else 'Even' for is_even in
26                  grouped.index]
```

[Terminal](#) [Test cases](#)[< PREV](#) [RESET](#) [SUBMIT](#) [NEXT >](#)

## 4.1.2. Dictionary to dataframe

25:19

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

**Create the DataFrame:**

- Convert the dictionary to a Pandas DataFrame.

**Add a new row:**

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

**Modify a row:**

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

**Delete a row:**

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

**Add a new column:**

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

**Modify a column:**

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

**Delete a column:**

- Remove the **Age** column.

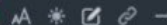
datafram...

```
63 # Adding a new row
64
65 new_name=input("New name: ")
66 new_age=int(input("New age: "))
67 new_row={'Name':
68 new_name, 'Age':new_age}
69 df=pd.concat([df,pd.DataFrame([new_row]),ignore_index=True)
70 # Display the DataFrame after adding a new row
71 print("After adding a row:\n",df)
72
73 # Modifying a row
74 modify_index=int(input("Index of row to modify: "))
75 new_age_mod=int(input("New age: "))
76 df.loc[modify_index,"Age"]=new_age_mod
77
78
79 # Display the DataFrame after modifying a row
80 print("After modifying a row:")
81 print(df)
82
83 # Deleting a row
84 delete_index=int(input("Index of row to delete: "))
85 df=df.drop(delete_index).reset_index(drop=True)
86 # Display the DataFrame after deleting a row
87 print("After deleting a row:")
88 print(df)
89
90 # Adding a new column
91 gender_input=input("Enter genders separated by space: ")
92 genders=gender_input.split()
```

[Terminal](#) [Test cases](#)

## 4.1.3. Student Information

15:42



Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

**Note:**

Refer to the displayed test cases for better understanding.

Sample Test Cases



Explorer

studentin...



SUBMIT

Debugger

```
1  """import pandas as pd
2
3  # Read the text file into a DataFrame
4  file = input()
5  data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6  "Grade"])
7
8  import pandas as pd
9
10 # Read the text file into a DataFrame
11 file = input()
12 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
13 "Grade"])
14
15 # write your code here..
16 print("First five rows:")
17 print(data.head(5))
18 # write your code here..
19 age=round(data['Age'].mean(),2)
20 print("Average age:", age)
21 print("Students with a grade up to B")
22 df=pd.DataFrame(data)
23 a=df[df['Grade']<='B']
24 print(a)
25
```

Terminal

Test cases

&lt; PREV

RESET

SUBMIT

NEXT &gt;



## 4.2.1. Month with the Highest Total Sales

01:58

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

## Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

## Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

+

monthFor...

SUBMIT

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # Convert Date column to datetime format
10 df['Date'] = pd.to_datetime(df['Date'])
11
12 # Create a new column 'Sales' = Quantity * Price
13 df['Sales'] = df['Quantity'] * df['Price']
14
15 # Extract month in YYYY-MM format
16 df['Month'] = df['Date'].dt.to_period('M').astype(str)
17
18 # Group the data by Month and calculate total sales for each month
19 monthly_sales = df.groupby('Month')['Sales'].sum()
20
21 # Find the month with the highest total sales
22 best_month = monthly_sales.idxmax()
23 highest_sales = monthly_sales.max()
24
25
26 # Find the month with the highest total sales
27
28
29 print(f"Best month: {best_month}")
```

Terminal Test cases

&lt; PREV RESET SUBMIT NEXT &gt;

## 4.2.2. Best Selling Product

01:28



Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

## Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

## Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases



monthFor...

SUBMIT

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # Group the data by 'Product' and sum up the 'Quantity'
10 product_sales = df.groupby('Product')['Quantity'].sum()
11
12 # Find the product with the highest total quantity sold
13 best_product = product_sales.idxmax()
14 highest_quantity = product_sales.max()
15 # Find the product with the highest total quantity sold
16
17 # Display the result
18 print(f"Best selling product: {best_product}")
19 print(f"Total quantity sold: {highest_quantity}")
20
```

Terminal Test cases

&lt; PREV RESET SUBMIT NEXT &gt;

## 4.2.3. City that Sold the Most Products

02:10     

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

## Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

## Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases 

Explorer

monthFor... 

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 # write the code..
10 # write the code..
11
12 # 3. Group the data by 'City' and calculate the total quantity sold for
13 # each city
14 # We select the 'Quantity' column and use .sum() on the groups
15 city_totals = df.groupby('City')['Quantity'].sum()
16
17 # 4. Find the city that sold the most products
18 # .idxmax() returns the index (the City name) of the maximum value in
19 # the series
20 best_city = city_totals.idxmax()
21
22
23 # Display the result
24 print(f"City sold the most products: {best_city}")
25
```

# Debugger

 Terminal  Test cases[< PREV](#) [RESET](#) [SUBMIT](#) [NEXT >](#)



## 4.2.4. Most Frequently Sold Product Pairs

01:42

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

## Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

## Explanation:

## Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

frequentl...

SUBMIT

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # write the code
12
13 # write the code
14
15 # Output the most frequent product pairs
16
17 # 1. Group the products by 'Date'.
18 # We create a list of products for each unique date.
19 grouped = df.groupby('Date')['Product'].apply(list)
20
21 # 2. Generate all possible pairs for each date.
22 # We sort the pairs (e.g., A,B instead of B,A) so they are counted
23 # together.
24 all_pairs = []
25 for products in grouped:
26     # combinations(products, 2) finds every pair in the list
27     for pair in combinations(sorted(products), 2):
28         all_pairs.append(pair)
29
```

Terminal Test cases

&lt; PREV RESET SUBMIT NEXT &gt;

## 4.2.5. Titanic Dataset Analysis and Data Cleaning

02:17

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

## Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2, 3101282,7.925,,S
```

titanicDat...

```
#3. Get the shape of the dataset
print(data.shape)

#4. Get a summary of the dataset (info)
print(data.info())

#5. Get basic statistics of the dataset
print(data.describe())

#6. Check for missing values
print(data.isnull().sum())

#7. Fill missing values in the 'Age' column with the median age
data['Age'] = data['Age'].fillna(data['Age'].median())

#8. Fill missing values in the 'Embarked' column with the mode
data['Embarked'] = data['Embarked'].fillna(data['Embarked'].mode()[0])

#9. Drop the 'Cabin' column due to many missing values
data = data.drop('Cabin', axis=1)

#10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
data['FamilySize'] = data['SibSp'] + data['Parch']

#10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

Terminal

Test cases

&lt; PREVIOUS

RESET

SUBMIT

NEXT &gt;



#### 4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

02:44

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

#### Sample Data:

```

PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
    
```

titanicDat...

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8
9 data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
10
11 # 2. Convert 'Sex' to numeric (male: 0, female: 1)
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode the 'Embarked' column
15 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
16
17 # 4. Get the mean age of passengers
18 mean_age = data['Age'].mean()
19 print(mean_age)
20
21 # 5. Get the median fare of passengers
22 median_fare = data['Fare'].median()
23 print(median_fare)
24
25 # 6. Get the number of passengers by class
26 pclass_counts = data['Pclass'].value_counts()
27 print(pclass_counts)
28
29 # 7. Get the number of passengers by gender
    
```

Terminal Test cases

[PREV](#) [RESET](#) [SUBMIT](#) [NEXT](#)

## 4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

07:30

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked\_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

## Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
```

titanicDat...

SUBMIT

```
55
56 #----Task Solutions----
57
58 #1. Calculate the survival rate by class
59 survival_rate_by_class = data.groupby('Pclass')['Survived'].mean()
60 print(survival_rate_by_class)
61
62 #2. Calculate the survival rate by embarkation location (Embarked_S)
63 survival_rate_by_embarked_s = data.groupby('Embarked_S')
64   ['Survived'].mean()
65 print(survival_rate_by_embarked_s)
66
67 #3. Calculate the survival rate by family size (FamilySize)
68 survival_rate_by_family_size = data.groupby('FamilySize')
69   ['Survived'].mean()
70 print(survival_rate_by_family_size)
71
72 #4. Calculate the survival rate by being alone (IsAlone)
73 survival_rate_by_is_alone = data.groupby('IsAlone')['Survived'].mean()
74 print(survival_rate_by_is_alone)
75
76 #5. Get the average fare by passenger class (Pclass)
77 avg_fare_by_class = data.groupby('Pclass')['Fare'].mean()
78 print(avg_fare_by_class)
79
80 #6. Get the average age by passenger class (Pclass)
81 avg_age_by_class = data.groupby('Pclass')['Age'].mean()
82 print(avg_age_by_class)
```

Terminal Test cases

&lt; PREV RESET SUBMIT NEXT &gt;



## 4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

02:09

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked\_S).
4. Get the number of non-survivors by embarkation location (Embarked\_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

| PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|-------------|----------|--------|------|-----|-----|-------|-------|--------|------|-------|----------|
|             |          |        |      |     |     |       |       |        |      |       |          |

## Sample Data:

```
PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
```

```
titanicDat...
data = pd.read_csv('titanic-dataset.csv')

# Preprocessing: Create dummy variables for 'Embarked'
# This creates 'Embarked_S' (1 if S, 0 otherwise) used in the requirements
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender (Sex)
survivors_by_gender = data[data['Survived'] == 1]['Sex'].value_counts()
print(survivors_by_gender)

# 2. Get the number of non-survivors by gender (Sex)
non_survivors_by_gender = data[data['Survived'] == 0]
['Sex'].value_counts()
print(non_survivors_by_gender)

# 3. Get the number of survivors by embarkation location (Embarked_S)
survivors_by_embarked_s = data[data['Survived'] == 1]
['Embarked_S'].value_counts()
print(survivors_by_embarked_s)

# 4. Get the number of non-survivors by embarkation location (Embarked_S)
non_survivors_by_embarked_s = data[data['Survived'] == 0]
['Embarked_S'].value_counts()
print(non_survivors_by_embarked_s)

# 5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
survived_children_pct = children['Survived'].mean()
print(survived_children_pct)
```

[Terminal](#) [Test cases](#)[< PREVIOUS](#) [RESET](#) [SUBMIT](#) [NEXT >](#)